

3.1.1 Source Files

- `process.c` : ELF 바이너리 파일 읽어서 시작
- `pagedir.c` : 메모리 관리하는 `page table` 관리.
(수정 X) 프로그램 메모리에서 뭘 꺼낼 때, 이 주소 진짜 써도 되는지 검사할 때 필요.

```
bool is_valid_user_ptr(const void *uaddr) {
    if (uaddr == NULL) return false;
    if (!is_user_vaddr(uaddr)) return false; // 유저 영역 주소? (vaddr < PHYS_BASE)
    struct thread *cur_thread = thread_current();
    if (pagedir_get_page(cur_thread->pagedir, uaddr) == NULL) return false; // physical address 매핑 안했다면 NULL
    return true;
}
```
- `syscall.c` : user process가 system call 할 때 handler를 사용해 system call 해줌.
- `exception.c` : 잘못된 접근일 때 메모리를 죽이고 메시지 출력
(수정 필요 됨)
- `gdt.c` : 메모리 다뤄는 segment 사용 => 정의하고 관리하는 표 (GDT)
(수정 X)
- `tss.c` : 프로그램이 커널로 넘어올 때 스택은 자동으로 바꿔주는 기능

3.1.2 File System

- No internal synchronization
- File size is fixed at creation time
- File data is allocated as a single extent
- No subdirectories
- File name are limited to 14 characters
- There is no file system repair tool anyway

• 3.1.4 Virtual Memory Layout

User virtual memory : 0 ~ PHY_BASE (0xc0000000) (3GB)

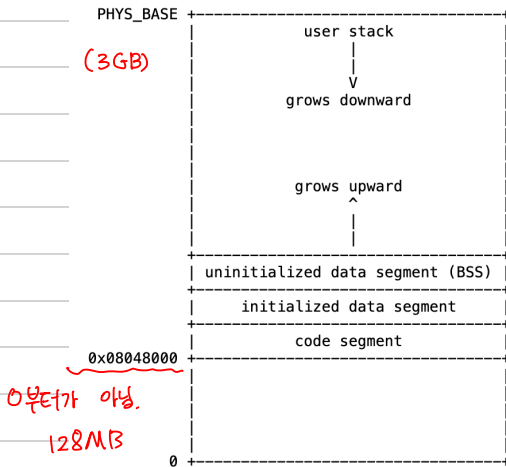
=> per-process. processor의 page directory base register 변경으로

사용자 가상 주소 전환됨.

thread 구조체에 uint32_t * pagedir; 포인터가 있음.

kernel virtual memory : PHY_BASE (3GB) ~ 4GB (virtual)
=> global함. 0 ~ 1GB (physical) } 1:1 매핑 (3GB 배정됨)

사용자 program은 kernel virtual memory 에 접근 불가



← 초기화 안된 전역변수 int a;

← 초기화 된 전역변수 int a=5;

← 실행 코드 main() { ... }, 함수{...}

• linker script : 컴파일할때 코드를 어디에 놓을지, 데이터를 어디에 놓을지 알려주는 파일.

3.1.5 Accessing User memory

- User가 이상한 pointer 주다면 프로그램 죽이고 OS는 살아있어야 함.

쉬운 방법: 미리 pointer 검사하고 쓰기

① 이 주소가 PHY_BASE 보다 작을까? (userprog/pagedir.c) is-user_vaddr(uaddr) { vaddr < PHY_BASE }

② 이 주소가 page table 에 매핑되어있을까? (threads/vaddr.c) pagedir_get_page(pagedir, uaddr)

* 조심: 죽일때 잠깐 lock 이나 malloc 메모리는 켜고 나옴.

3.3.3 Argument Passing , 3.5 , 3.5.1

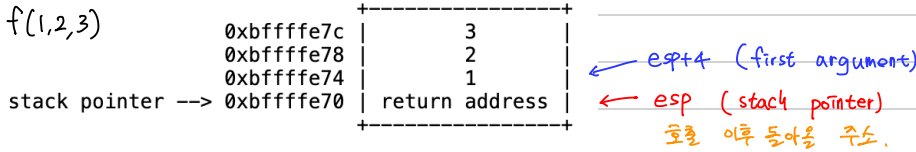
⇒ process_execute 고쳐서 program name, argument 넘기기

프(어쓰기) 여러개도 하나로 처리함. lib/string.c 에 strtok_r()

1. argument stack에 push (caller → callee)

오른쪽 → 왼쪽 순서로 push (esp 4씩 감소)

아래 방향으로 stack 성장.



2. CALL f [① caller return address 스택에 push
② 함수 f로 jump 하여 제어 넘김.

3. callee (불러온 함수) 에서 return 값이 있다면 EAX 레지스터에 저장

4. 함수 종료시 RET 명령어로 return address pop하고 그 주소로 jump

5. return하면 caller가 다시 stack에 push했던거 pop해줄.

• User program 시작

```
void _start (int 4argc, char *argv[]) {
    exit (main (argc, argv));
}
```

fake return address 0xbffffcc

ex) /bin/ls -l foo bar

arg[0] arg[1] arg[2] arg[3] , argc=4 (argument count)

Address	Name	Data	Type
0xc0000000	(기타 PHYS-BASE)	10 포함해서	
0xbfffffc	argv[3][...]	"bar\0"	char[4]
0xbfffff8	argv[2][...]	"foo\0"	char[4]
0xbfffff5	argv[1][...]	"-\0" 3바이트	char[3]
0xbfffffd	argv[0][...]	"/bin/ls\0"	char[8]
0xbffffec	word-align	0	uint8_t
0xbffffe8	argv[4]	0	char *
0xbffffe4	argv[3]	0xbfffffc	char *
0xbffffe0	argv[2]	0xbfffff8	char *
0xbffffdc	argv[1]	0xbfffff5	char *
0xbffffd8	argv[0]	0xbfffffd	char *
0xbffffd4	argv	0xbffffd8	char **
0xbffffd0	argc	4	int
0xbffffcc	return address	0	void (*)()

① 10을포함한 문자열 역순으로 저장하면서 10 포함 크기만큼 내리기
② 성능을 위해 주소를 4의 배수로 맞춤. (padding byte)
③ argv 배열 구성하기 (각각 char *)
: 문자열 포인터들을 역순으로 push
④ argv (char **) : argv[0]의 주소
⑤ argc : argument 개수
⑥ fake return address

최종 esp

hex-dump (esp, size) ⇒ 디버깅 도구

3.3.4 System Calls

```
void
syscall_init (void)
{
    intr_register_int (0x30, 3, INTR_ON, syscall_handler, "syscall");

    // ㉠㉡㉢㉣ - 파일 작업 락 초기화
    lock_init (&file_lock);
}
```

syscall handler (struct intr_frame *f UNUSED)

f → esp : system call number (= 함수명)

f → eax : syscall return value 넣는곳

☆ 포인터가 PHY_BASE 아래엔지.
점심적인 주소인지

uaddr < PHYS_BASE

pagedir = get_page (thread → pagedir, uaddr)
= NULL 이면 physical address 애당 X.

```
// 컴퓨터를 꺼버리는 시스템 콜
// shutdown_power_off() 함수를 호출해서 Pintos를 완전히 종료
// 주의: 이걸 호출하면 deadlock이나 에러 정보가 남지 않기 때문에, 정말 필요할 때만
void halt(void) {
    shutdown_power_off();
}

// 현재 스레드를 종료하고, 종료 상태를 반환
// 만약 부모 프로세스가 자식 프로세스를 wait()하고 있었다면, 이 status 값을 부모가 받아감
// 0 = 성공 / 0이 아닌 값 = 실패
void exit(int status) {
    struct thread *cur_thread = thread_current();
    cur_thread->is_exit = status;
    printf("%s: exit(%d)\n", cur_thread->name, status);
    thread_exit();
}

// 성공하면 true, 실패하면 false
bool create(const char *file, unsigned initial_size) {
    if (file == NULL) exit(-1);
    return filesys_create(file, initial_size);
}
```

```
// 성공하면 새로운 파일 디스크립터(fd)를 반환, 실패하면 -1
int open(const char *file) {
    lock_acquire(&file_lock); // → 파일 시스템 동시접근 X
    struct file *f = filesys_open(file);
    if (f == NULL) {
        lock_release(&file_lock); // 이거 안하면 deadlock
        return -1;
    }

    struct thread *cur_thread = thread_current();

    // fd = 0(STDIN_FILENO)은 표준 입력, fd = 1(STDOUT_FILENO)은 표준 출력, 실패하면 -1
    // fd_table index 늘리면서 file descriptor 리턴해야하니까 기록
    int fd = cur_thread->fd_idx++;

    if (fd >= FD_MAX) {
        printf("File descriptor table is full\n"); // 디버깅용
        file_close(f);
        lock_release(&file_lock);
        return -1;
    }

    cur_thread->fd_table[fd] = f; // 파일 저장

    lock_release(&file_lock);
    return fd;
}
```

• Thread 73세 내 file descriptor table

```
// ㉠㉡㉢㉣ - File Descriptor Table
int fd_idx; // 0 = stdin, 1 = stdout, 2 = file descriptor 시작
struct file *fd_table[FD_MAX]; // fd_table[0] = stdin, fd_table[1] = stdout
```

2부터 시작

```

// 파일이나 콘솔로 데이터를 쓰기
int write(int fd, const void *buffer, unsigned size) {
    // 지금 Project 2-1에서는 write()를 완전히 다 구현할 필요는 없다.
    // 딱 'fd == 1' (콘솔 출력)만 제대로 하면 된다.
    if (fd == 1) {
        putchar(buffer, size);
        return size;
    }
    // 2-2에서 구현 예정
    else {
        return 0;
    }
}

// 파일 디스크립터(fd)를 닫기
// fd가 표준 입출력(STDIN, STDOUT)이면 무시
// fd가 범위 밖에 있으면 무시
// 유효한 fd라면 해당 파일을 닫고 fd_table 엔트리를 NULL로 초기화
void close(int fd) {
    if (fd == 0 || fd == 1) return; // 표준 입출력
    if (fd < 0 || fd >= FD_MAX) {
        // printf("[close] File descriptor error: %d\n", fd); // 디버깅용
        return;
    }

    struct thread *cur_thread = thread_current();
    file_close (cur_thread->fd_table[fd]);
    cur_thread->fd_table[fd] = NULL;
}

```